Green AI for Object Detection: An Energy-Aware Two-Tier YOLO26 People Detector

Dinakar Nayak N | Student ID: 259055047 | dnnm1@student.le.ac.uk
CO7091 Computational Intelligence and Software Engineering – Problem Solving
University of Leicester | December 2026

1. Introduction

Object detection locates and labels objects in images. It underpins pedestrian safety, crowd monitoring and search-and-rescue drones. Yet detectors are usually compared on accuracy alone, although larger models consume more energy on every frame, and cameras often run continuously on limited power. Schwartz et al. [1] criticise this “Red AI” trend and argue that efficiency should be reported alongside accuracy. I therefore propose an energy-aware people detector: a lightweight model handles most frames, a larger model is called only when needed, and the system measures its own energy, making the accuracy–energy trade-off explicit.

(93 words)

2. Proposed Technology

Overview. The system is a two-tier cascade (Appendix B). Like the Viola–Jones attentional cascade [4], it spends little computation on easy inputs and more only where necessary.

Tier 1: lightweight detector. Every frame is processed by YOLO26n (2.4M parameters, 5.5 GFLOPs) [3], restricted to the COCO “person” class. YOLO26 removes Distribution Focal Loss and offers optional NMS-free inference, suiting CPU and edge devices. Compressed YOLO variants have shown large energy savings with little accuracy loss [5].

Tier 2: reliability-triggered escalation. YOLO26s (9.5M parameters, 20.9 GFLOPs) re-processes a frame only when Tier 1 looks unreliable: a person detected with confidence 0.25–0.50, or a weak candidate (0.10–0.25) that Tier 1 may have missed. On video, every 30th frame is also escalated as a safety check, and optional brightness and blur checks flag poor images. This follows energy-aware edge routing such as ECORE [6]. Because most frames are easy, average cost approaches the nano model’s while accuracy approaches the small model’s.

Energy measurement. CodeCarbon [7] tracks CPU, GPU and RAM power, converting energy to CO_2e using regional grid carbon intensity [8]. Inference and model-loading energy are recorded separately and reported beside accuracy, following Green AI practice [1, 2].

Evaluation. Five configurations were tested on the same 128 COCO images [9] (Appendix A). The proposed system matched YOLO26s’s recall (0.740) and nearly matched its AP50 (0.822 vs 0.826) while using 39% less energy per image. An ablation showed that the candidate trigger recovered missed people, whereas scene checks added cost without benefit on this dataset.

Implementation. The system uses Python, Ultralytics and OpenCV and runs live from a webcam (Appendix C).

(267 words)

3. Conclusion

This report proposed a Green AI people detector that pairs a lightweight YOLO26n with a reliability-triggered YOLO26s and measures its own energy using CodeCarbon. In testing, it matched YOLO26s’s recall while cutting energy per image by 39%. Challenges remain: people missed entirely by YOLO26n are caught only by periodic checks, CodeCarbon often estimates rather than measures power, and training energy is not counted. Future work will apply INT8 quantisation and knowledge

distillation, learn escalation thresholds from data, and validate the system on a Raspberry Pi with a hardware power meter.

(90 words)

References

[1] Schwartz, R., Dodge, J., Smith, N.A. and Etzioni, O. (2020) ‘Green AI’, Communications of the ACM, 63(12), pp. 54–63. doi: 10.1145/3381831.

[2] Strubell, E., Ganesh, A. and McCallum, A. (2019) ‘Energy and policy considerations for deep learning in NLP’, Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 3645–3650. arXiv: 1906.02243.

[3] Ultralytics (2026) Ultralytics YOLO26 documentation. Available at: https://docs.ultralytics.com/models/yolo26/ (Accessed: 4 November 2026).

[4] Viola, P. and Jones, M. (2001) ‘Rapid object detection using a boosted cascade of simple features’, Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), vol. 1, pp. 511–518.

[5] Humes, E., Navardi, M. and Mohsenin, T. (2023) ‘Squeezed Edge YOLO: Onboard object detection on edge devices’, arXiv: 2312.11716.

[6] Alqahtani, D.K., Rodriguez, M.A., Cheema, M.A., Rezatofighi, H. and Toosi, A.N. (2025) ‘ECORE: Energy-conscious optimized routing for deep learning models at the edge’, arXiv: 2507.06011.

[7] CodeCarbon contributors (2024) CodeCarbon: track and reduce CO2 emissions from your computing. Available at: https://github.com/mlco2/codecarbon (Accessed: 4 November 2026).

[8] Lacoste, A., Luccioni, A., Schmidt, V. and Dandres, T. (2019) ‘Quantifying the carbon emissions of machine learning’, arXiv: 1910.09700.

[9] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C.L. (2014) ‘Microsoft COCO: Common objects in context’, European Conference on Computer Vision (ECCV), pp. 740–755.

Appendix A: Experimental Results

Five configurations were run by benchmark.py on the same 128 images of the COCO128 validation set [9], which contain 254 annotated people. Each two-tier row adds one escalation trigger, so the table is also an ablation study. Accuracy is COCO-style average precision for the person class at IoU 0.5 (AP50) and averaged over IoU 0.5–0.95 (AP50–95). These AP values come from the project's own lightweight evaluator in benchmark.py and are approximations, not official COCO (pycocotools) results. Recall is measured at confidence 0.25 and IoU 0.5. Energy was measured with CodeCarbon [7] over each run and divided by the number of images; a warm-up image was excluded. Periodic escalation is disabled here because the test images are independent rather than video frames.

Configuration	AP50	AP50–95	Recall	ms / img	J / img	Escal. %
YOLO26s only (baseline)	0.826	0.615	0.740	210.5	2.558	–
YOLO26n only	0.776	0.539	0.661	82.9	0.972	–
Two-tier: band only	0.824	0.601	0.728	125.1	1.466	20.3
Two-tier: band + candidates (proposed)	**0.822**	**0.601**	**0.740**	**133.6**	**1.566**	**25.0**
Two-tier: band + candidates + scene	0.822	0.602	0.740	147.9	1.735	27.3

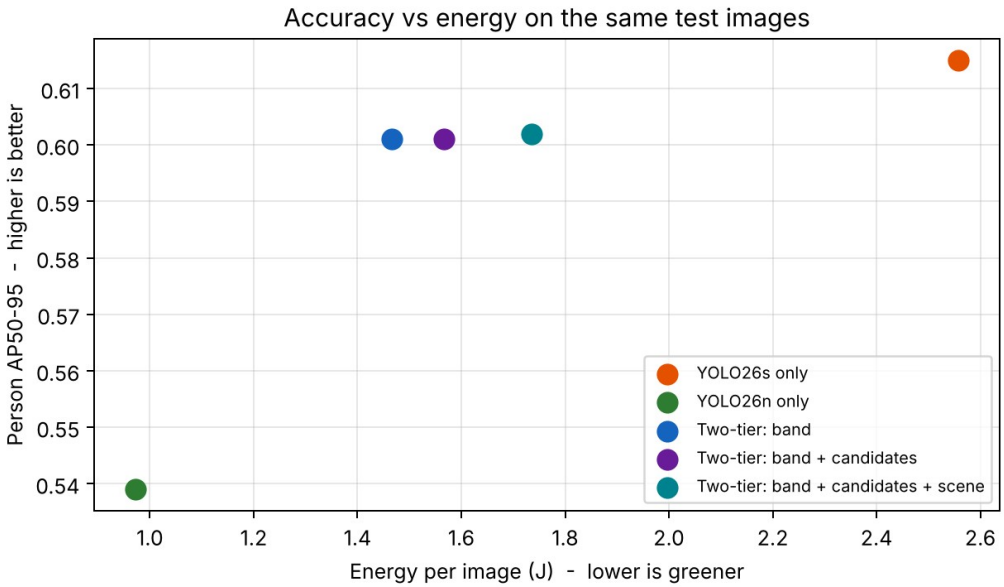


Figure 1. Person AP50–95 against energy per image for each configuration.

Key findings

- The proposed configuration matched the YOLO26s baseline's recall (0.740), kept 99.5% of its AP50 (0.822 vs 0.826) and 97.7% of its AP50–95 (0.601 vs 0.615), and used 39% less energy per image (1.566 J vs 2.558 J). It was also 37% faster (133.6 ms vs 210.5 ms).
- Escalating on confidence band alone was cheapest (1.466 J) but missed some people (recall 0.728). Adding the weak-candidate trigger escalated six more images and restored recall to the baseline level for 7% more energy, which addresses the risk that YOLO26n misses a person entirely.
- Brightness and blur checks escalated three further images but gave no measurable accuracy gain on COCO128, which contains few dark or blurred images. They are therefore optional (--scene) and need testing on low-light footage.

- YOLO26n alone was the most efficient (0.972 J per image) but lost 5.0 points of AP50 and 7.9 points of recall, showing why escalation is needed for difficult scenes.

Model loading cost

Keeping both models in memory costs little. Loading YOLO26n and YOLO26s together took 0.13 s and 1.5 J, which is less than the energy of processing one image, and the weight files occupy 5.4 MB and 19.9 MB respectively.

Test conditions and limitations

Tests ran on a 2-vCPU Linux cloud machine without a GPU, using Ultralytics 8.4 and CodeCarbon 3.3. On machines without hardware power counters, CodeCarbon estimates CPU power from the processor's rated thermal design power, so absolute joule values are estimates; the relative comparison between configurations, run on the same machine, is more reliable. CO_2e is not compared here because it depends on the electricity grid where the code runs; the demonstration system prints CO_2e for the local grid. COCO128 is a small sample, so a larger test set including low-light and crowded scenes would give tighter estimates.

Appendix B: System Architecture

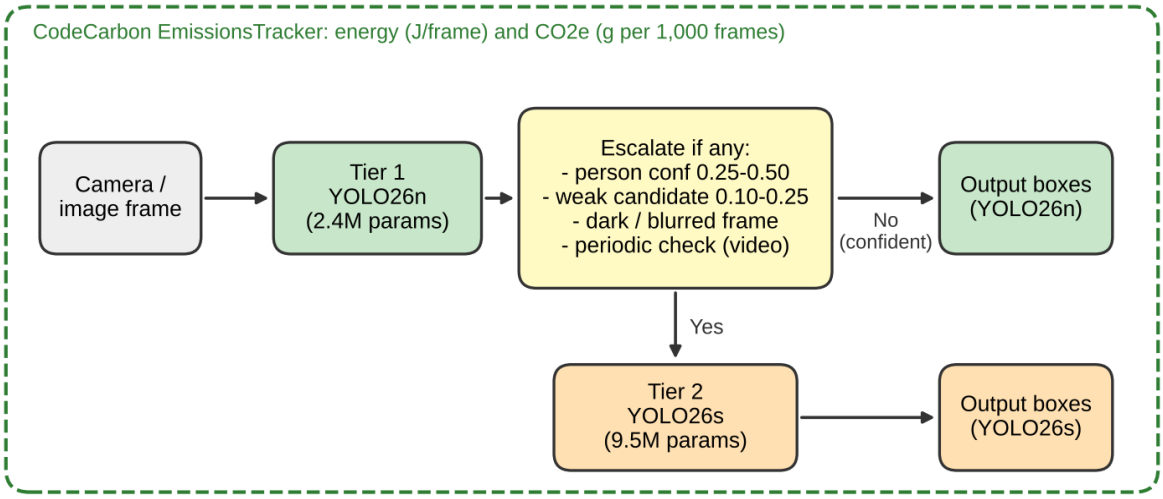


Figure 2. Energy-aware two-tier detection pipeline. CodeCarbon measures the energy of the whole pipeline.

Each frame is first processed by YOLO26n. The frame is re-processed by YOLO26s, and its output used instead, if any of four triggers fires: (1) a person detected with confidence in the uncertain band [0.25, 0.50); (2) a weak candidate with confidence in [0.10, 0.25), suggesting a person YOLO26n may have missed; (3) optionally, the frame is very dark (mean brightness below 40) or blurred (Laplacian variance below 30); (4) on video, every 30th frame, as a periodic safety check. Otherwise the YOLO26n output is accepted. Each threshold is a command-line option that trades energy for accuracy.

Model specifications (from Ultralytics [3])

YOLO26n: 2.4M parameters, 5.5 GFLOPs, 40.9 mAP50–95 on COCO (all classes). YOLO26s: 9.5M parameters, 20.9 GFLOPs, 48.6 mAP50–95. YOLO26s therefore needs roughly four times the computation of YOLO26n per image.

Appendix C: Implementation and Demonstration

The system is implemented in Python using Ultralytics (YOLO26), OpenCV and CodeCarbon. Source files:

- green_detector.py – the two-tier detector and live webcam/video/image demo. The display shows which model handled each frame and why it was escalated. The summary reports inference and end-to-end FPS, energy per frame, CO_2e and model-loading energy.
- benchmark.py – reproduces Appendix A (accuracy, recall, time and energy for all five configurations) and draws Figure 1.
- requirements.txt and README.md – installation and usage instructions.

Installation: pip install -r requirements.txt. Live demonstration: python green_detector.py runs the adaptive detector on the webcam. Green boxes indicate frames handled by YOLO26n and orange boxes indicate frames escalated to YOLO26s. Running with --mode small and --mode nano gives the baselines, and each run appends its energy per frame, FPS and escalation rate to run_log.csv for comparison.

Core escalation logic

```
boxes, confs = run(YOLO26n, frame, conf=0.10)
if   any(0.25 <= c < 0.50 for c in confs): reason = 'uncertain'
elif any(0.10 <= c < 0.25 for c in confs): reason = 'candidate'
elif scene_checks and (dark or blurred):   reason = 'scene'
elif frame_number % 30 == 0:               reason = 'periodic'
if reason:
    boxes, confs = run(YOLO26s, frame)       # escalate
return boxes[confs >= 0.25], confs[confs >= 0.25]
```